

SERVICIA

Plataforma de Aseguramiento Contable

Piloto Batia

Documentación Técnica y Funcional

Análisis de arquitectura, flujos operativos, modelos de datos
e instrucciones para Unidad de Inteligencia y Contabilidad

Emitido por: Dirección General

Clasificación: Confidencial · Uso interno

Versión: 2.0

Fecha: Abril 2026

Stack: Flask 3.0 · SQLAlchemy · PostgreSQL · Cloud Run

Destinatarios: Unidad de Inteligencia (UI) · Área de Contabilidad · Tesorería · Dirección

Índice

1. Resumen Ejecutivo	3
2. Arquitectura del Sistema	4
2.1. Stack Tecnológico	4
2.2. Estructura del Proyecto	4
2.3. Application Factory y Blueprints	5
2.4. Seguridad y Autenticación	5
3. Modelo de Datos	7
3.1. Entidades Principales	7
3.1.1. Role y User	7
3.1.2. Provider y AuthorizedAccount	7
3.1.3. Folio — El Expediente Central	8
3.1.4. FolioWorkflow — Los 8 Pasos	8
3.2. Entidades de Control Fiscal	8
3.2.1. ProviderCompliance	9
3.2.2. CfdiRecord	9
3.2.3. BankMovement y ReconciliationAlert	9
3.3. Diagrama de Relaciones	9
4. Flujo Operativo de 8 Pasos	11
4.1. Lógica de Recálculo Automático	11
4.2. Estatus Semáforo del Folio	12
5. Módulos del Sistema	13
5.1. Módulo de Autenticación (/auth)	13
5.2. Módulo Principal (/)	13
5.2.1. Dashboard Ejecutivo	13
5.2.2. Búsqueda Global (/busqueda)	13
5.3. Módulo de Folios (/folios)	13
5.3.1. Creación de un Folio	13
5.4. Módulo Operativo (/operacion)	14
5.4.1. Cumplimiento SAT/EFOS	14
5.4.2. Entregables por Tipo de Proveedor	14

5.4.3. Traspasos SPEI	14
5.4.4. CFDI	15
5.4.5. Conciliación Bancaria	15
5.4.6. Presupuestos	15
5.4.7. Reportes Mensuales	15
5.4.8. Acuses de Conformidad	16
6. Responsabilidades Operativas por Área	17
6.1. Unidad de Inteligencia (UI)	17
6.2. Área de Contabilidad	17
6.3. Conciliaciones Críticas Mensuales	17
7. Sistema de Alertas y Auditoría	18
7.1. Centro de Alertas	18
7.2. Log de Auditoría	18
8. Cálculo de Fechas Límite y KPI de Entregables	20
9. Datos de Demostración	21
9.1. Usuarios Demo	21
9.2. Proveedores y Folios Demo	21
10.Despliegue y Operación	22
10.1. Entornos	22
10.2. Dockerfile	22
10.3. Cloud Run — Configuración	22
10.4. Variables de Entorno	22
10.5. Instalación en Desarrollo	22
11.Reglas Absolutas del Sistema	24
12.Diseño de Interfaz	25
12.1. Paleta de Colores	25
12.2. Componentes Principales	25
13.Acuse de Recibo	26

1. Resumen Ejecutivo

Descripción General

Servicia es una plataforma web de aseguramiento contable desarrollada en Flask (Python) para **Batia**, empresa de gestión de servicios integrales. El sistema digitaliza y controla el ciclo completo de vida de cada contratación registrada en SINGA: desde la apertura del expediente hasta el cierre fiscal verificado por Contabilidad, pasando por validaciones SAT/EFOS, traspasos SPEI, vinculación de CFDI y conciliación bancaria.

El proyecto responde a una necesidad crítica: garantizar que **toda operación de pago esté completamente materializada**, documentada y conciliada, reduciendo el riesgo fiscal ante el SAT y permitiendo que cualquier auditoría interna o externa pueda verificar en minutos la legalidad y trazabilidad de cada transacción.

Características principales

- **Flujo operativo de 8 pasos** con bloqueos secuenciales: ningún paso puede ejecutarse sin que el anterior esté completado.
- **Gestión de folios SINGA** con checklist de entregables diferenciado según tipo de proveedor (outsourcing, materiales, servicios).
- **Control SAT/EFOS**: bloquea pagos a proveedores sin Opinión de Cumplimiento vigente.
- **Módulo de conciliación bancaria** con carga de PDF y extracción automática de movimientos.
- **Sistema de alertas automáticas** (depósito sin folio, monto excedido, cuenta no autorizada, diferencia CFDI-banco).
- **Tablero ejecutivo** personalizado por rol con KPIs en tiempo real.
- **Despliegue en Google Cloud Run** con base de datos PostgreSQL gestionada.

2. Arquitectura del Sistema

2.1. Stack Tecnológico

Capa	Tecnología
Lenguaje	Python 3.11
Framework Web	Flask 3.0.3
ORM	Flask-SQLAlchemy 3.1.1
Migraciones	Flask-Migrate 4.0.7 (Alembic)
Autenticación	Flask-JWT-Extended 4.6.0 + sesiones de servidor
Templates	Jinja2 (server-side rendering)
BD Desarrollo	SQLite (instance/servicia.db)
BD Producción	PostgreSQL (Cloud SQL, GCP)
Servidor WSGI	Gunicorn 23.0.0 (1 worker, 8 threads, timeout 120s)
Contenedor	Docker (Python 3.11-slim)
Plataforma Cloud	Google Cloud Run + Cloud Build
Excel	openpyxl 3.1.5
PDF parsing	pypdf 5.4.0

Tabla 1: Stack tecnológico de Servicia

2.2. Estructura del Proyecto

El proyecto sigue el patrón *Application Factory* de Flask con *blueprints* modulares. Cada área funcional tiene su propio blueprint con sus rutas y plantillas.

```

contabilidad/
  app/
    __init__.py      <- Application factory
    config.py        <- Variables de entorno
    extensions.py    <- Registro de db, migrate, jwt
    models.py        <- 18 modelos SQLAlchemy
    audit.py         <- Log de auditoria
    seed.py          <- Datos demo idempotentes
    auth/routes.py   <- Login, logout, JWT
    main/routes.py   <- Landing, dashboard, busqueda
    folios/routes.py <- CRUD de folios SINGA
    operations/
      routes.py      <- SAT, CFDI, traspasos, conciliacion
      services.py    <- Logica de negocio y workflow
    static/
      css/styles.css <- Hoja de estilos
      js/main.js     <- Interacciones frontend
    templates/       <- Plantillas Jinja2 por modulo
  instance/servicia.db <- BD SQLite (desarrollo)
  ref/              <- Manual de usuario y referencias
  uploads/          <- Archivos subidos por usuarios
  run.py            <- Entrypoint desarrollo
  wsgi.py           <- Entrypoint produccion
  Dockerfile
  cloudbuild.yaml   <- Pipeline CI/CD en GCP

```

```
requirements.txt
```

Listing 1: Árbol de directorios del proyecto

2.3. Application Factory y Blueprints

La función `create_app()` en `app/__init__.py` sigue el patrón *factory* para permitir múltiples instancias (testing, desarrollo, producción):

```
1 def create_app(config_class=Config):
2     app = Flask(__name__)
3     app.config.from_object(config_class)
4
5     # Inicializacion de extensiones
6     db.init_app(app)
7     migrate.init_app(app, db)
8     jwt.init_app(app)
9
10    # Registro de blueprints
11    app.register_blueprint(auth_bp, url_prefix='/auth')
12    app.register_blueprint(main_bp)
13    app.register_blueprint(folios_bp, url_prefix='/folios')
14    app.register_blueprint(operations_bp, url_prefix='/operacion')
15
16    # Headers de seguridad en cada respuesta
17    @app.after_request
18    def security_headers(response):
19        response.headers['X-Content-Type-Options'] = 'nosniff'
20        response.headers['X-Frame-Options'] = 'DENY'
21        return response
22
23    return app
```

Listing 2: Registro de blueprints en `create_app()`

2.4. Seguridad y Autenticación

El sistema implementa autenticación dual:

- **Sesiones de servidor** (Flask session): para el acceso vía navegador. El decorador `@login_required` verifica la presencia de `user_id` en sesión.
- **JWT (JSON Web Tokens)**: endpoint de API POST `/auth/api/token`, que emite tokens con claims de rol y nombre.

```
1 def roles_required(*allowed_roles):
2     def decorator(f):
3         @wraps(f)
4         def wrapper(*args, **kwargs):
5             role = session.get('role')
6             if role not in allowed_roles:
7                 flash('No tienes permisos para esta accion.', 'error')
8                 return redirect(url_for('main.dashboard'))
```

```
9         return f(*args, **kwargs)
10     return wrapper
11 return decorator
```

Listing 3: Decorador de control de acceso por rol

3. Modelo de Datos

La base de datos contiene **18 modelos SQLAlchemy** que se mapean a las tablas de la BD. A continuación se describen los más importantes.

3.1. Entidades Principales

3.1.1. Role y User

```

1 class User(db.Model):
2     __tablename__ = 'users'
3     id            = db.Column(db.Integer, primary_key=True)
4     full_name     = db.Column(db.String(120), nullable=False)
5     email         = db.Column(db.String(120), unique=True,
6                               nullable=False)
7     password_hash = db.Column(db.String(256), nullable=False)
8     role_id       = db.Column(db.Integer, db.ForeignKey('roles.id'))
9     is_active     = db.Column(db.Boolean, default=True)
10    role           = db.relationship('Role', lazy='joined')
11
12    def set_password(self, raw):
13        self.password_hash = generate_password_hash(raw)
14
15    def check_password(self, raw):
16        return check_password_hash(self.password_hash, raw)

```

Listing 4: Modelo User con hash de contraseña

Los roles disponibles son: direccion, ui, contabilidad, tesoreria y auditor.

3.1.2. Provider y AuthorizedAccount

```

1 class Provider(db.Model):
2     __tablename__ = 'providers'
3     id            = db.Column(db.Integer, primary_key=True)
4     name          = db.Column(db.String(200), unique=True,
5                               nullable=False)
6     provider_type = db.Column(db.String(30)) # outsourcing,
7                                             materiales, servicios
8     sat_status    = db.Column(db.String(30))
9     sat_valid_until = db.Column(db.Date)
10
11 class AuthorizedAccount(db.Model):
12     __tablename__ = 'authorized_accounts'
13     clabe         = db.Column(db.String(18), unique=True, nullable=False)
14     rfc           = db.Column(db.String(13))
15     is_active     = db.Column(db.Boolean, default=True)
16     __table_args__ = (
17         db.CheckConstraint("LENGTH(clabe) = 18"),)

```

Listing 5: Modelos Provider y AuthorizedAccount

3.1.3. Folio — El Expediente Central

El modelo Folio representa cada contratación registrada en SINGA. Es la entidad pivote: todos los demás registros operativos apuntan a un folio.

```

1 class Folio(db.Model):
2     __tablename__ = 'folios'
3     id = db.Column(db.Integer,
4         primary_key=True)
5     singa_number = db.Column(db.String(50), unique=True,
6         nullable=False)
7     provider_id = db.Column(db.Integer,
8         db.ForeignKey('providers.id'))
9     provider_type = db.Column(db.String(30))
10    communication_responsible = db.Column(db.String(120))
11    contract_amount = db.Column(db.Numeric(14, 2))
12    budget_amount = db.Column(db.Numeric(14, 2))
13    status = db.Column(db.String(30),
14        default='pendiente')
15    created_at = db.Column(db.DateTime,
16        default=datetime.utcnow)
17
18    #status: pendiente / en_proceso / listo_para_cierre / cerrado /
19    #critico
20    __table_args__ = (
21        db.CheckConstraint("status IN ('pendiente','en_proceso',"
22            "'listo_para_cierre','cerrado','alerta','critico')"),
23        db.CheckConstraint("budget_amount > 0"),
24    )

```

Listing 6: Modelo Folio con estatus semáforo

3.1.4. FolioWorkflow — Los 8 Pasos

```

1 class FolioWorkflow(db.Model):
2     __tablename__ = 'folio_workflows'
3     folio_id = db.Column(db.Integer,
4         db.ForeignKey('folios.id'),
5         unique=True)
6     step_1_ui_folio = db.Column(db.Boolean, default=False)
7     step_2_contab_sat = db.Column(db.Boolean, default=False)
8     step_3_ui_deliverables = db.Column(db.Boolean, default=False)
9     step_4_tesoreria_transfer = db.Column(db.Boolean, default=False)
10    step_5_contab_cfdi = db.Column(db.Boolean, default=False)
11    step_6_contab_reconciliation = db.Column(db.Boolean,
12        default=False)
13    step_7_ui_ready_to_close = db.Column(db.Boolean, default=False)
14    step_8_contab_closed = db.Column(db.Boolean, default=False)
15    updated_at = db.Column(db.DateTime,
16        onupdate=datetime.utcnow)

```

Listing 7: Modelo FolioWorkflow

3.2. Entidades de Control Fiscal

3.2.1. ProviderCompliance

Almacena el estado de cumplimiento SAT/EFOS de cada proveedor con relación 1:1.

Campo	Descripción
sat_opinion_status	no_cargada / vigente / vencida
sat_valid_until	Fecha de vencimiento de la opinión SAT
efos_flag	Aparece en lista EFOS del SAT (booleano)
activity_match	Actividad económica congruente con el servicio
rfc_matches_account	RFC coincide con CLABE registrada
notes	Observaciones de Contabilidad

Tabla 2: Campos de ProviderCompliance

3.2.2. CfdiRecord

```

1 class CfdiRecord(db.Model):
2     __tablename__ = 'cfdi_records'
3     uuid          = db.Column(db.String(36), unique=True, nullable=False)
4     xml_ref       = db.Column(db.String(255))      # ruta del XML subido
5     pdf_ref       = db.Column(db.String(255))      # ruta del PDF subido
6     amount        = db.Column(db.Numeric(14, 2))
7     issued_at     = db.Column(db.Date)
8     status        = db.Column(db.String(20), default='vigente')
9     # status: vigente / cancelado

```

Listing 8: Modelo CfdiRecord

3.2.3. BankMovement y ReconciliationAlert

ReconciliationAlert — Tipos de alerta automática	
deposito_sin_folio	Depósito bancario sin folio SINGA asociado
excede_presupuesto	Monto de traspaso o movimiento supera el presupuesto
cuenta_no_autorizada	Movimiento a cuenta no registrada en la plataforma
diferencia_cfdi_banco	Diferencia entre CFDI registrado y movimiento bancario
Severidades: media / alta	
Estatus: abierta / resuelta	

Tabla 3: Tipos de alerta de conciliación

3.3. Diagrama de Relaciones

Role	1 ---< N	User
Provider	1 ---< N	AuthorizedAccount
Provider	1 ---< N	Folio
Provider	1 --- 1	ProviderCompliance

Folio	1	---	1	FolioWorkflow
Folio	1	---	< N	FolioDeliverableItem
Folio	1	---	< N	Transfer
Folio	1	---	< N	CfdiRecord
Folio	1	---	< N	BankMovement
Folio	1	---	< N	ReconciliationAlert
Folio	1	---	< N	BudgetConfig
AuthorizedAccount	1	---	< N	Transfer
EvidenceFile	N	---	1	Folio
EvidenceFile	N	---	1	FolioDeliverableItem (opcional)
AuditLog	N	---	1	User (nullable)
MonthlyDirectionReport	N	---	1	User

Listing 9: Relaciones entre entidades principales

4. Flujo Operativo de 8 Pasos

Principio fundamental

Ningún paso puede saltarse. El sistema aplica bloqueos secuenciales en código: intentar registrar un traspaso sin que SAT esté validado genera un error de negocio inmediato. El workflow avanza solo cuando el paso anterior está completo.

#	Área	Acción en la plataforma	Desbloquea
1	UI	Registrar folio SINGA con proveedor, tipo y responsable	Apertura del expediente
2	Contabilidad	Verificar proveedor en SAT y subir Opinión de Cumplimiento	Cuenta autorizada para traspaso
3	UI	Subir entregables operativos según tipo de proveedor	Expediente pasa a “En proceso”
4	Tesorería	Registrar traspaso SPEI con folio bancario	Conciliación del monto
5	Contabilidad	Vincular CFDI al folio y validar monto vs presupuesto	Sistema verifica excedente
6	Contabilidad	Conciliar con estado de cuenta PDF	Semáforo del Dashboard se actualiza
7	UI	Confirmar que todos los entregables están completos	Folio pasa a “Listo para cierre”
8	Contabilidad	Revisar y aprobar expediente completo	Folio marcado “Cerrado”

4.1. Lógica de Recálculo Automático

La función `recalculate_workflow_from_data()` en `services.py` implementa las reglas de avance de cada paso:

```

1 def recalculate_workflow_from_data(folio, workflow, deliverables,
2                                   has_transfer, has_cfdi,
3                                   has_reconciliation):
4     # Paso 1: siempre True (folio ya existe)
5     workflow.step_1_ui_folio = True
6
7     # Paso 2: SAT/EFOS vigente
8     compliance = ProviderCompliance.query.filter_by(
9         provider_id=folio.provider_id).first()
10    workflow.step_2_contab_sat = (
11        compliance and
12        compliance.sat_opinion_status == 'vigente' and
13        not compliance.efos_flag
14    )
15
16    # Paso 3: todos los entregables UI subidos
17    ui_items = [d for d in deliverables if d.owner_area == 'ui']

```

```

18 workflow.step_3_ui_deliverables = (
19     bool(ui_items) and all(d.uploaded for d in ui_items)
20 )
21
22 # Paso 4: traspaso solo si SAT validado
23 workflow.step_4_tesoreria_transfer = (
24     workflow.step_2_contab_sat and has_transfer
25 )
26
27 # Paso 5: CFDI solo si traspaso realizado
28 workflow.step_5_contab_cfdi = (
29     workflow.step_4_tesoreria_transfer and has_cfdi
30 )
31
32 # Paso 6: conciliacion solo si CFDI vigente
33 workflow.step_6_contab_reconciliation = (
34     workflow.step_5_contab_cfdi and has_reconciliation
35 )
36
37 # Paso 7: listo cierre si UI completo y conciliacion completa
38 workflow.step_7_ui_ready_to_close = (
39     workflow.step_3_ui_deliverables and
40     workflow.step_6_contab_reconciliation
41 )
42
43 # Paso 8: cerrado si listo + conciliacion
44 workflow.step_8_contab_closed = (
45     workflow.step_7_ui_ready_to_close and
46     workflow.step_6_contab_reconciliation
47 )

```

Listing 10: Recalculo del workflow – services.py

4.2. Estatus Semáforo del Folio

Estatus	Condición
critico	Existen alertas de severidad “alta” abiertas
cerrado	Paso 8 completo
listo_para_cierre	Pasos 7 y 6 completos
en_proceso	Alguno de los pasos 3, 4 o 5 completo
pendiente	Estado inicial (ningún paso avanzado)

Tabla 5: Estatus semáforo de los folios SINGA

5. Módulos del Sistema

5.1. Módulo de Autenticación (/auth)

- **GET/POST /auth/login:** Valida email y contraseña (bcrypt vía Werkzeug), crea sesión con `user_id`, `user_name` y `role`. Registra evento en `AuditLog`.
- **POST /auth/logout:** Limpia sesión y redirige al landing.
- **POST /auth/api/token:** Emite JWT con claims `role` y `name` para consumo de API externa.

5.2. Módulo Principal (/)

5.2.1. Dashboard Ejecutivo

El dashboard se personaliza según el rol del usuario autenticado:

Rol	Información en el Dashboard
UI	Entregables pendientes, folios activos, folios críticos
Contabilidad	Entregables contables, alertas abiertas, folios sin CFDI
Tesorería	Folios por traspasar, alertas de severidad alta
Dirección	Reportes mensuales, alertas antiguas (+5 días), acuses firmados
Auditor	Vista completa de KPIs sin permisos de escritura

Tabla 6: Información personalizada por rol en el dashboard

5.2.2. Búsqueda Global (/busqueda)

Permite buscar por texto libre. Devuelve hasta 25 resultados por categoría: folios (SINGA, responsable, proveedor), proveedores, UUIDs de CFDI, referencias SPEI y alertas.

5.3. Módulo de Folios (/folios)

Vista *master-detail*: lista de folios con filtros en el panel izquierdo y detalle del folio seleccionado en el panel derecho (workflow, entregables, transferencias, CFDIs, movimientos, alertas, evidencias y eventos).

5.3.1. Creación de un Folio

Al crear un nuevo folio, el sistema:

1. Valida que el número SINGA sea único.
2. Verifica que el proveedor tenga al menos una **cuenta bancaria autorizada activa**. Si no la tiene, bloquea la operación.

3. Valida que el presupuesto no supere el monto del contrato.
4. Crea el registro **FolioWorkflow** (8 pasos en falso).
5. Genera automáticamente el checklist de **FolioDeliverableItem** según el tipo de proveedor.

5.4. Módulo Operativo (/operacion)

5.4.1. Cumplimiento SAT/EFOS

Contabilidad gestiona el estado fiscal de cada proveedor. Al actualizar:

- Se registra la Opinión de Cumplimiento con su fecha de vencimiento.
- Se marcan flags: EFOS, congruencia de actividad, coincidencia RFC-CLABE.
- El sistema **recalcula automáticamente el estatus de todos los folios** activos del proveedor.

Regla crítica de negocio

No es posible registrar un traspaso SPEI si el proveedor tiene `efos_flag = True`, opinión SAT **vencida** o **no_cargada**, o si la actividad económica no coincide. El sistema retorna un error de negocio con mensaje específico.

5.4.2. Entregables por Tipo de Proveedor

Tipo	Entregables adicionales
Outsourcing	Lista de personal, turnos, asistencias firmadas, bitácoras operativas, evidencia fotográfica, supervisión documentada, relación de insumos
Materiales	Fotos de materiales recibidos, acuse de recepción firmado, registro de inventario, evidencia de entrega al destino final
Servicios	Definición de procesos, reportes con KPIs, evidencia de intervención (logs/tickets), resultados documentados, acta de entrega-recepción

Tabla 7: Entregables específicos por tipo de proveedor

Todos los tipos comparten los **entregables base**: orden de compra firmada, contrato vigente, responsable de comunicación y correos formales del período.

5.4.3. Traspasos SPEI

El registro de un traspaso ejecuta **5 validaciones secuenciales**:

1. Paso 2 (SAT) debe estar completo.

2. La cuenta bancaria debe pertenecer al proveedor del folio.
3. SAT vigente, EFOS negativo, actividad y RFC validados.
4. Opinión SAT no vencida a la fecha.
5. Monto $\leq$ presupuesto autorizado (o requiere aprobación de Dirección).

Si el monto supera el presupuesto y hay aprobación de Dirección, el traspaso se registra pero **genera automáticamente una alerta** de tipo `excede_presupuesto` con severidad alta.

5.4.4. CFDI

Vincula comprobantes fiscales digitales a cada folio. Permite subir los archivos XML y PDF del CFDI. Requiere que el Paso 4 (Traspaso) esté completo. Registra UUID, monto, fecha de emisión y estatus (vigente/cancelado). Las cancelaciones deben incluir motivo documentado.

5.4.5. Conciliación Bancaria

Dos modalidades:

1. **Carga de PDF:** El sistema analiza el estado de cuenta bancario con `pypdf` y extrae movimientos mediante expresiones regulares. Crea alertas automáticas para cada anomalía detectada.
2. **Registro manual:** El operador captura movimientos individuales. Se aplican 4 reglas de alerta automáticas:
 - Depósito sin folio $\rightarrow$ alerta alta `deposito_sin_folio`
 - Monto $>$ presupuesto $\rightarrow$ alerta alta `excede_presupuesto`
 - Cuenta no autorizada $\rightarrow$ alerta alta `cuenta_no_autorizada`
 - Sin CFDI vigente $\rightarrow$ alerta media `diferencia_cfdi_banco`

5.4.6. Presupuestos

Carga de presupuestos autorizados por período (YYYY-MM). Dos modalidades: carga masiva desde Excel con columnas `folio`, `presupuesto autorizado`, `periodo`; o alta manual individual por folio.

5.4.7. Reportes Mensuales

Generación automática de reporte ejecutivo con KPIs calculados:

- Porcentaje de cierre de folios en el período

- Alertas abiertas y alertas críticas
- Número de proveedores con SAT vigente
- Ratio de cumplimiento SAT sobre total de proveedores

5.4.8. Acuses de Conformidad

Registro digital de la firma de conformidad de dos áreas: UI y Contabilidad. Incluye nombre del firmante, fecha y notas. Visible en el dashboard de Dirección.

6. Responsabilidades Operativas por Área

Esta sección está basada en la *Carta de Instrucciones Operativas* emitida por Dirección General para UI y Contabilidad.

6.1. Unidad de Inteligencia (UI)

La UI es el área responsable de construir y mantener el expediente operativo de cada folio SINGA. **Sin la evidencia generada por UI, no existe sustento para la factura ni para el pago.**

Acción	Plazo máximo	KPI
Registrar folio SINGA en la plataforma	Mismo día de apertura	% en tiempo
Subir orden de compra y contrato	48 horas de iniciado	% OC subidas
Subir evidencia operativa del período	Antes del día 25	% expedientes completos
Subir asistencias y bitácoras	Máx. 5 días hábiles	% asistencias
Subir evidencia fotográfica	Máx. 5 días hábiles	% fotos subidas
Comunicaciones relevantes del mes	Antes del cierre	% comunicaciones
Marcar folio listo para revisión	Día 27 de cada mes	% folios enviados

Tabla 8: KPIs de la Unidad de Inteligencia

6.2. Área de Contabilidad

Acción	Plazo máximo	KPI
Subir opinión SAT antes de autorizar pago	Previo al traspaso	% opiniones previas
Subir Excel de presupuestos SINGA	Día 5 de cada mes	% meses con presupuesto
Correr conciliación bancaria	Días 28-30 del mes	% conciliaciones
Documentar y resolver alertas	5 días hábiles	% alertas en plazo
Vincular CFDIs a folios SINGA	Máx. 3 días de emitido	% CFDIs vinculados
Entregar reporte mensual a Dirección	Día 5 del mes siguiente	% reportes a tiempo
Verificar proveedores en lista EFOS	Antes del primer pago	% verificados

Tabla 9: KPIs del Área de Contabilidad

6.3. Conciliaciones Críticas Mensuales

Contabilidad debe realizar y documentar **tres cruces de conciliación** antes del cierre mensual:

Conciliación	Qué se cruza	Entregable
Bancos vs CFDIs vs Contabilidad	Cada depósito/retiro debe tener CFDI y asiento contable. Sin diferencias.	Reporte conciliación IA
IVA acreditable vs IVA trasladado	IVA de facturas recibidas vs IVA cobrado.	Reporte fiscal mensual
Nómina vs IMSS vs CFDIs	CFDIs de nómina coinciden con pagos SUA y nómina contabilizada.	Constancias SUA

Tabla 10: Conciliaciones mensuales obligatorias

7. Sistema de Alertas y Auditoría

7.1. Centro de Alertas

Las alertas se generan automáticamente por el motor de conciliación y las validaciones de traspasos. Cada alerta tiene severidad (**media/alta**) y estatus (**abierta/resuelta**).

Regla de resolución

Toda alerta abierta debe resolverse en un plazo máximo de **5 días hábiles**. Las alertas de severidad alta que superen este plazo aparecen destacadas en el dashboard de Dirección. Un folio con alertas altas abiertas cambia automáticamente su estatus a **crítico**.

7.2. Log de Auditoría

El modelo `AuditLog` registra **toda acción importante** en el sistema. La función `log_action()` en `audit.py` se invoca desde todas las rutas de escritura:

```

1 def log_action(*, user_id, action, entity, entity_id=None,
2   details=None):
3     """
4     Crea un registro de auditoria en la base de datos.
5     Solo usa argumentos nombrados (keyword-only).
6     """
7     entry = AuditLog(
8         user_id=user_id,
9         action=action,          # ej: 'folio_creado',
10        'traspaso_registrado'
11        entity=entity,          # ej: 'Folio', 'Transfer',
12        'ProviderCompliance'
13        entity_id=entity_id,
14        details=details         # JSON o texto libre con contexto
15    )
16    db.session.add(entry)
17    db.session.commit()

```

Listing 11: Función de log de auditoría

Eventos auditados: login/logout, creación de folios, actualización de SAT, registro de traspasos, vinculación de CFDI, carga de presupuestos, generación de reportes.

8. Cálculo de Fechas Límite y KPI de Entregables

El módulo `services.py` implementa la lógica de cálculo de plazos y el estado KPI de cada entregable.

```
1 def due_date_for_code(code, owner_area):
2     today = date.today()
3     if code in ('oc', 'contrato'):
4         return add_business_days(today, 2)
5     elif code in ('asistencias', 'fotos', 'bitacoras', 'insumos',
6                 'acuse_recepcion', 'inventario',
7                 'evidencia_entrega',
8                 'procesos', 'kpis', 'logs', 'resultados'):
9         return add_business_days(today, 5)
10    elif code == 'comunicaciones':
11        return month_cutoff(25)      # dia 25 del mes en curso
12    elif code in ('sat', 'efos'):
13        return add_business_days(today, 1)
14    elif code == 'cfdi':
15        return add_business_days(today, 3)
16    elif code == 'conciliacion':
17        return month_cutoff(30)      # dia 30 del mes en curso
18    return add_business_days(today, 5)
19
20 def kpi_status_for_item(item):
21     if item.uploaded:
22         return 'completo'
23     if item.due_date and item.due_date < date.today():
24         return 'vencido'
25     return 'pendiente'
```

Listing 12: Cálculo de fechas límite por tipo de entregable

9. Datos de Demostración

El archivo `app/seed.py` carga datos de demostración de forma **idempotente** (no duplica registros si ya existen).

9.1. Usuarios Demo

Email	Rol	Contraseña
saló@batia.local	Dirección	servicia2026
mgonzalez@batia.local	Tesorería	servicia2026
lhernandez@batia.local	Tesorería	servicia2026
rfuentes@batia.local	UI	servicia2026
cmorales@batia.local	UI	servicia2026
pramirez@batia.local	Contabilidad	servicia2026

Tabla 11: Usuarios de demostración

9.2. Proveedores y Folios Demo

- **Limpiadores SA** (outsourcing) — con cuentas Banorte y BBVA
- **Insumos Alfa** (materiales) — con cuenta BBVA
- **TechServ** (servicios) — con cuenta Banorte
- **Mant. Delta** (outsourcing) — con cuenta Banorte

Tres folios demo con estatus variables para prueba de todas las funcionalidades.

10. Despliegue y Operación

10.1. Entornos

Entorno	Base de datos	Servidor
Desarrollo	SQLite (instance/servicia.db)	python run.py
Producción	PostgreSQL (Cloud SQL, GCP)	Gunicorn en Cloud Run

10.2. Dockerfile

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV PORT=8080

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

CMD ["sh", "-c", \
    "gunicorn --bind 0.0.0.0:${PORT} \
        --workers 1 --threads 8 --timeout 120 wsgi:app"]
```

Listing 13: Dockerfile de producción

10.3. Cloud Run — Configuración

Parámetro	Valor
Instancias mínimas	0 (scale-to-zero)
Instancias máximas	3
CPU	1 vCPU
Memoria	512 MiB
Timeout	120 segundos
Cuenta de servicio	cloud-run-mlh@...iam.gserviceaccount.com
Región	us-central1

Tabla 12: Configuración de Cloud Run

10.4. Variables de Entorno

10.5. Instalación en Desarrollo

Variable	Propósito
SECRET_KEY	Clave de sesiones Flask
JWT_SECRET_KEY	Clave para firma de tokens JWT
DATABASE_URL	Cadena de conexión (SQLite o PostgreSQL)
COMPANY_NAME	Nombre de la empresa para branding
UPLOAD_FOLDER	Ruta local para archivos subidos
MAX_CONTENT_LENGTH	Tamaño máximo de upload (16 MB)

Tabla 13: Variables de entorno

```
# 1. Instalar dependencias
pip install -r requirements.txt

# 2. Configurar entorno
cp .env.example .env
# Editar .env con los valores correspondientes

# 3. Iniciar aplicacion (auto-seed incluido)
python run.py

# 4. Acceder a la plataforma
# http://127.0.0.1:5000
```

Listing 14: Pasos de instalación local

11. Reglas Absolutas del Sistema

Las siguientes reglas están implementadas en código como bloqueos duros y no tienen excepciones operativas:

Reglas absolutas — Aplicables a todas las áreas

1. No se registra ningún folio sin el número de SINGA correspondiente.
2. No se puede traspasar dinero a una cuenta que no esté autorizada por Dirección en la plataforma.
3. No se emite ni se paga ninguna factura sin contrato firmado previo.
4. No se deduce ningún gasto sin materialización completa del servicio.
5. No se paga a ningún proveedor sin Opinión de Cumplimiento SAT vigente y cargada.
6. Todo debe estar conciliado: **Banco ↔ CFDI ↔ Contabilidad ↔ Folio SINGA**.
7. Cualquier diferencia entre el depósito y el presupuesto SINGA requiere autorización escrita de Dirección antes de proceder.
8. Las alertas generadas por el sistema deben resolverse en máximo **5 días hábiles**.

Riesgo fiscal crítico

Pagar a un proveedor que aparezca en la lista EFOS (Empresas que Facturan Operaciones Simuladas) **invalida automáticamente la deducción fiscal** y genera responsabilidad solidaria para la empresa. Servicia bloquea este escenario en código: un proveedor con `efos_flag = True` no puede recibir traspasos.

12. Diseño de Interfaz

12.1. Paleta de Colores

Variable	Uso	Hex
ink-900	Fondo oscuro, textos primarios	#0b1736
sky-500	Acento principal, links, headers	#1c78c0
teal-500	Acento secundario, subtítulos	#0f8d8f
lime-500	Estados ok, éxito	#5f9d2f
amber-500	Advertencias, estado pendiente	#c97a1a
rose-500	Errores, estados críticos	#c94a56
paper-100	Fondo de tarjetas y panels	#f8fafc

Tabla 14: Sistema de colores de la interfaz

12.2. Componentes Principales

- **Dashboard de rol:** Hero personalizado con KPIs focalizados y accesos rápidos.
- **Master-Detail de folios:** Lista filtrable con panel de detalle contextual.
- **Workflow visual:** Indicadores de los 8 pasos con estado completado/pendiente.
- **Alertas flash:** Auto-cierre en 5.2 segundos con animación.
- **Efectos visuales:** Orbs flotantes animados en el fondo (CSS keyframes).
- **Reveal al scroll:** Intersection Observer para animar elementos al entrar en viewport.

13. Acuse de Recibo

Unidad de Inteligencia (UI)	Área de Contabilidad
_____ Nombre y firma del responsable Fecha: _____	_____ Nombre y firma del responsable Fecha: _____

Este documento es de uso interno y confidencial.
Servicia · Módulo de Aseguramiento Contable · Batia

Generado a partir del análisis del código fuente
y la Carta de Instrucciones Operativas oficial.

Versión 2.0 · Abril 2026